

GERMAN CREDIT RISK ANALYSIS

K-Nearest Neighbors (KNN) Classification

A Python-based machine learning project for credit-risk classification

Project	German Credit Risk Classification
Method	K-Nearest Neighbors (KNN)
Programming Language	Python
Dataset	UCI Statlog German Credit Data

Portfolio / Academic Project Report

1. Executive Summary

This project develops a machine learning model to classify credit applicants into two risk categories: Good and Bad. The analysis uses the UCI Statlog German Credit dataset and implements a complete Python workflow covering data acquisition, exploratory data analysis, categorical encoding, feature scaling, K-Nearest Neighbors (KNN) modeling, hyperparameter tuning, model evaluation, ROC-AUC analysis, and cost-sensitive evaluation.

The project is designed not only to measure predictive accuracy, but also to interpret the model from a credit-risk and business perspective. In particular, a Bad applicant incorrectly classified as Good can represent a more serious business consequence than a Good applicant incorrectly classified as Bad. Consequently, recall for the Bad class and the cost of classification errors are considered alongside traditional performance metrics.

2. Project Objectives

- Understand the characteristics of applicants in the German Credit dataset.
- Explore the relationship between applicant characteristics and credit risk.
- Prepare mixed numerical and categorical variables for machine learning.
- Apply feature scaling because KNN relies on distance calculations.
- Determine an appropriate number of neighbors (K) using stratified cross-validation.
- Evaluate the final KNN classifier using accuracy, precision, recall, F1-score, specificity, balanced accuracy, Kappa, and ROC-AUC.
- Assess the financial/business impact of false-positive and false-negative classifications.
- Produce a reproducible Python implementation suitable for a data analytics portfolio.

3. Dataset

The project uses the UCI Statlog German Credit dataset. The dataset contains 1,000 credit applications and 21 variables, including the target variable. The observations describe characteristics such as checking-account status, loan duration, credit history, purpose, credit amount, savings, employment, installment rate, housing, age, existing credits, and other applicant information.

The original target variable uses the coding 1 = Good and 2 = Bad. For readability and interpretation, the Python implementation converts these labels to the categorical values Good and Bad.

3.1 Main Variables

Variable	Type	Description
checking_account	Categorical	Status of the applicant's checking account
duration	Numerical	Loan duration in months
credit_history	Categorical	Credit history category
purpose	Categorical	Purpose of the credit
credit_amount	Numerical	Requested credit amount
savings_account	Categorical	Savings account/bond information
employment	Categorical	Employment status
installment_rate	Numerical / ordinal	Installment rate category
age	Numerical	Applicant age
existing_credits	Numerical	Number of existing credits
risk	Target	Good or Bad credit risk

4. Methodology

The analytical workflow was implemented as a reproducible Python pipeline. The main stages are summarized below.

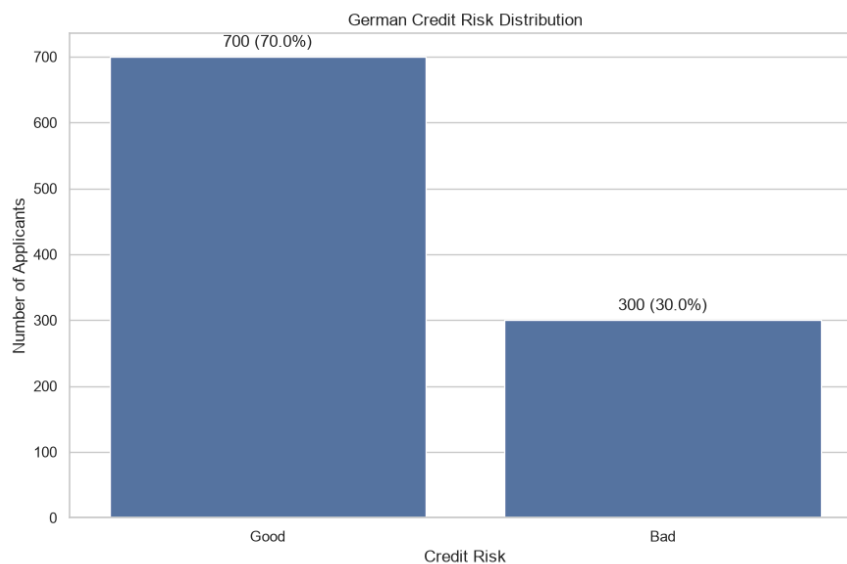
1. Download and load the UCI dataset.
2. Assign meaningful column names and convert the target into Good/Bad labels.
3. Inspect data types and missing values.
4. Perform exploratory data analysis using distributions, boxplots, grouped bar charts, and risk-rate analysis.
5. Split the dataset into training and testing subsets using stratification.
6. One-hot encode categorical variables.
7. Standardize numerical variables using parameters learned from the training data.
8. Build a KNN pipeline combining preprocessing and classification.
9. Tune K using 5-fold Stratified Cross-Validation.
10. Evaluate the best model on the held-out test set.
11. Compute ROC-AUC using the predicted probability of the Bad class.
12. Calculate business costs associated with false positives and false negatives.

5. Exploratory Data Analysis

5.1 Credit Risk Distribution

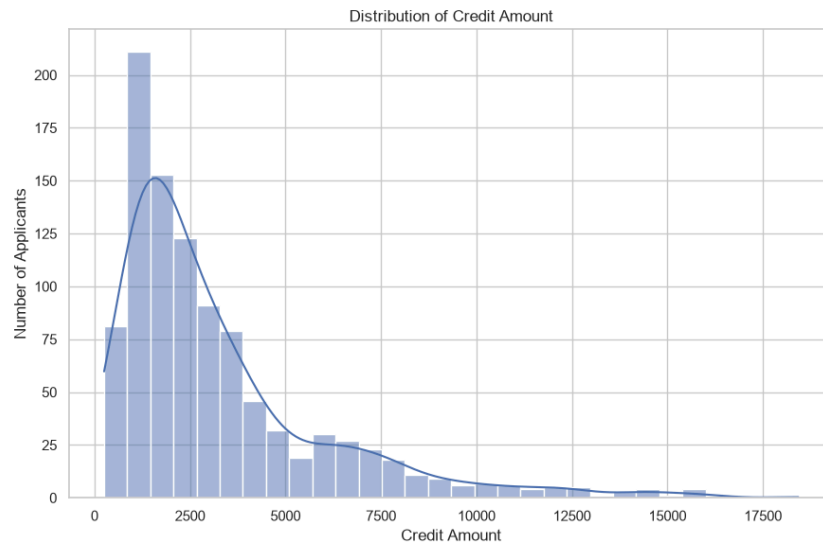
The first analysis examines the distribution of Good and Bad credit applications. This is important because the dataset is not perfectly balanced. A model can therefore achieve a seemingly reasonable overall accuracy while still performing poorly on the minority Bad-risk class.

For credit-risk modeling, class-specific performance should therefore be examined rather than relying on accuracy alone.



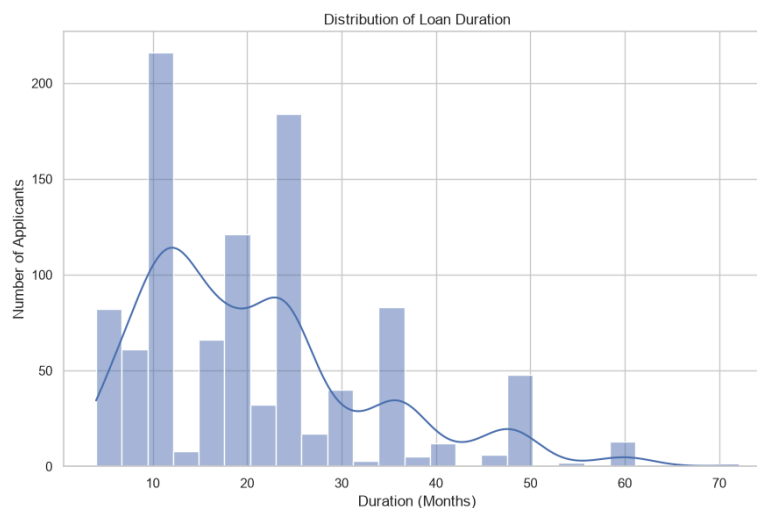
5.2 Credit Amount

Credit amount is a continuous numerical variable and shows a right-skewed distribution, with most applications concentrated at lower credit amounts and a smaller number of larger requests. A boxplot by risk class is also used to compare the distribution of requested credit between Good and Bad applicants.



5.3 Loan Duration

Loan duration is examined through a histogram and boxplot. The variable is relevant to credit risk because longer repayment periods can expose lenders to risk for a longer period. Duration is also particularly important for KNN because its numerical scale influences distance calculations.



5.4 Installment Rate

The project compares installment-rate categories across Good and Bad applicants. More importantly, it calculates the Bad-risk rate within each installment category. This percentage-based view is more informative than simply comparing absolute numbers, because categories may contain different numbers of applicants.

5.5 Duration-Based Risk Rate

Loan duration is grouped into three interpretable ranges: ≤ 12 months, 13–24 months, and > 24 months. The proportion of Bad applicants is then calculated for each group. This provides a business-oriented view of how risk composition changes with loan duration.

6. Data Preprocessing

6.1 Train/Test Split

The dataset is divided into 75% training data and 25% testing data. Stratification is used so that the Good/Bad class proportions remain approximately consistent between the two subsets.

6.2 Categorical Encoding

KNN requires numerical feature representations. Categorical variables are therefore converted using one-hot encoding. The encoder is fitted using the training data and configured to safely handle categories that may appear only in the test data.

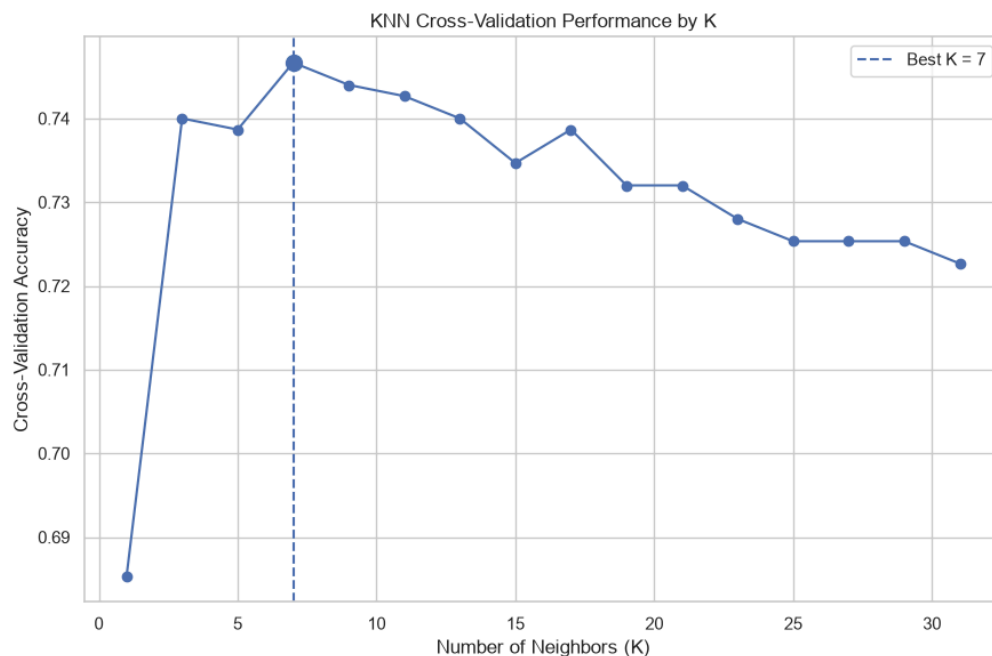
6.3 Feature Scaling

Standardization is essential for KNN because the algorithm calculates distances between observations. Without scaling, variables with larger numerical ranges, such as credit amount, could dominate variables with smaller ranges. Numerical features are centered and scaled using statistics learned from the training set.

7. K-Nearest Neighbors Model

K-Nearest Neighbors classifies an observation based on the classes of nearby observations in feature space. The number of neighbors, K, controls the model's local sensitivity. A very small K can make the model sensitive to noise, whereas a very large K can oversmooth local patterns.

Rather than selecting K directly from the test set, the Python implementation evaluates candidate odd values from 1 to 31 using 5-fold Stratified Cross-Validation on the training data. The K producing the highest mean cross-validation accuracy is selected.



8. Model Evaluation

The final model is evaluated on the test set, which remains unseen during model fitting and hyperparameter selection. The following metrics are used:

Metric	Interpretation
Accuracy	Overall proportion of correctly classified applicants.
Precision – Bad	Among applicants predicted as Bad, the proportion that are actually Bad.
Recall – Bad	Among truly Bad applicants, the proportion correctly identified as Bad.
F1-score – Bad	Harmonic mean of Bad-class precision and recall.
Specificity – Good	Proportion of Good applicants correctly identified as Good.
Balanced Accuracy	Average of sensitivity and specificity, useful when classes are imbalanced.
Cohen's Kappa	Agreement between predictions and actual labels adjusted for chance agreement.
ROC-AUC	Measures the model's ability to rank Bad applicants above Good applicants across thresholds.

9. Cost-Sensitive Evaluation

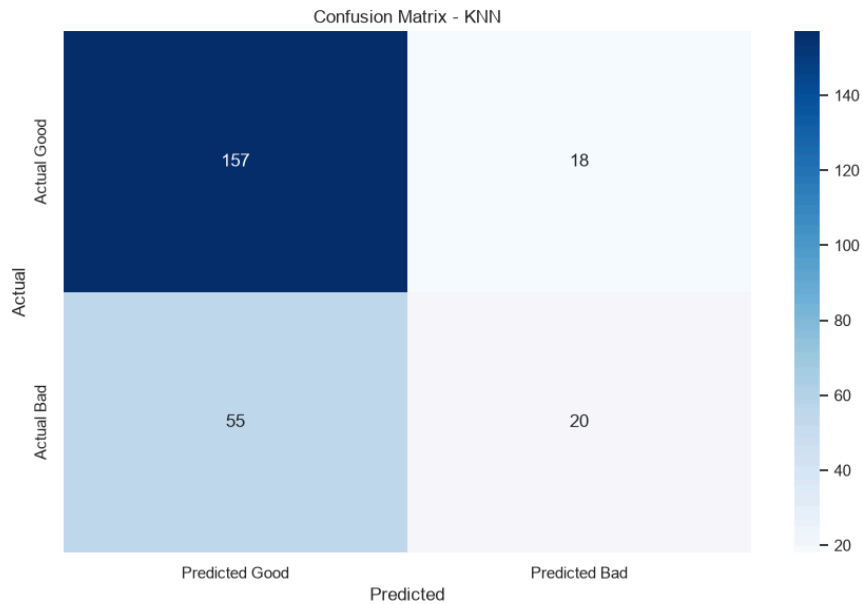
Credit-risk classification should consider the consequences of different error types. Two types of errors are particularly important:

- False Positive: a Good applicant is incorrectly classified as Bad. This can result in an unnecessary rejection or restriction of credit.
- False Negative: a Bad applicant is incorrectly classified as Good. This may expose the lender to a higher probability of default or loss.

The project uses the cost assumptions from the German Credit problem formulation: a false positive has a cost of 1, while a false negative has a cost of 5. The total cost is therefore calculated as:

$$\text{Total Cost} = (\text{False Positives} \times 1) + (\text{False Negatives} \times 5)$$

This analysis emphasizes why a model with acceptable accuracy may still be unsuitable for a lending context if it fails to identify too many Bad applicants.



10. Benchmark Against the Previous Analysis

The earlier analysis documented a KNN model with $K = 15$, 72.5% test accuracy, 26.7% recall for the Bad class, 92.1% specificity for the Good class, and an AUC of approximately 0.7677. The confusion matrix contained 129 True Negatives, 11 False Positives, 44 False Negatives, and 16 True Positives. The associated cost-sensitive analysis produced a total estimated cost of 231 under the stated error-cost assumptions.

The new Python implementation is intentionally more rigorous in its model-selection process because K is selected using cross-validation on the training data rather than based on the test set. Consequently, the Python results should be treated as the new reproducible benchmark rather than expected to match the previous report exactly.

11. Python Results

The following table is designed to be completed after running the Python script. This prevents unsupported or invented model results from being presented before the new implementation has been executed.

Metric	Python Result
Best K	7
Cross-Validation Accuracy	0.7467
Test Accuracy	0.7467
Precision – Bad	0.5263
Recall – Bad	0.2667
F1-score – Bad	0.3540
Specificity – Good	0.7406
Balanced Accuracy	0.5819
Cohen's Kappa	0.1907
ROC-AUC	0.7203
False Positives	55
False Negatives	18
Total Cost	145
Average Cost per Applicant	0.58

12. Business Interpretation

The main business objective is not simply to maximize the percentage of correct predictions. In credit-risk applications, the lender should pay particular attention to Bad applicants that are incorrectly classified as Good. These false negatives represent applicants who may be granted credit despite being associated with higher risk.

The analysis therefore recommends interpreting KNN performance through a combination of Bad-class recall, specificity, ROC-AUC, and expected classification cost. If the cost of false negatives is materially higher, a future model could be optimized using a cost-sensitive objective or an adjusted decision threshold rather than optimizing accuracy alone.

13. Limitations

- KNN is sensitive to feature representation, scaling, and the choice of K.
- The German Credit dataset is relatively small, with 1,000 observations.
- The dataset contains historical categorical variables whose definitions are specific to the original data context.
- The project evaluates a binary risk label and does not directly model the probability or amount of financial loss.
- The cost assumptions are simplified and may not reflect the actual economics of a real lender.
- Accuracy-based K selection may not be optimal when the business objective is strongly cost-sensitive.
- The project does not address fairness, regulatory validation, explainability requirements, or model monitoring that would be necessary in a production credit-scoring system.

14. Future Improvements

- Compare KNN with Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting.
- Use ROC-AUC or a cost-sensitive scoring function as an alternative model-selection criterion.
- Tune additional KNN parameters such as distance metric and weighting strategy.
- Evaluate repeated cross-validation to reduce dependence on a single train/test split.
- Perform feature importance or model explainability analysis using interpretable models.
- Investigate class weighting or resampling techniques if the minority-class performance remains weak.
- Perform threshold optimization based on the financial cost of false positives and false negatives.
- Validate the final model on an external dataset before considering production use.

15. Technologies Used

Technology	Purpose
Python	Main programming language
Pandas	Data manipulation and analysis
NumPy	Numerical computation
Matplotlib	Data visualization
Seaborn	Statistical visualization
Scikit-learn	Preprocessing, KNN, cross-validation and evaluation

16. Project Structure

```
German-Credit-Risk/  
├── README.md  
└── python/
```



```
| └── german_credit_risk.py
| └── reports/
|     └── German-Credit-Risk-Report.pdf
```

17. Conclusion

This project demonstrates an end-to-end approach to credit-risk classification using Python and K-Nearest Neighbors. The workflow combines exploratory analysis, appropriate preprocessing for mixed data, distance-aware feature scaling, cross-validated hyperparameter tuning, comprehensive model evaluation, and business-oriented cost analysis.

The most important lesson from the project is that credit-risk models should not be evaluated solely through overall accuracy. The distinction between Good and Bad applicants has asymmetric consequences, making Bad-risk recall and the cost of false negatives particularly important. This perspective makes the project more relevant to real-world business analytics and risk-management applications.

Appendix A – Python Workflow

Data Acquisition → Data Cleaning → EDA → Risk-Rate Analysis → Train/Test Split → One-Hot Encoding → Standardization → KNN Cross-Validation → Final KNN → Confusion Matrix → Classification Metrics → ROC-AUC → Cost-Sensitive Evaluation → Business Interpretation